Bending Spoons

Subscription Product Analytics

Project Specification

Yahia Ghallale
Graduate Data Scientist position

25 September 2026

Project demonstration prepared as part of the application.
All data and results in this document are synthetic and do not describe Bending Spoons.

1 Problem Statement

Subscription products generate signals across events, users, subscriptions, billing, acquisition, and experiments. Without a consistent data foundation, teams risk making decisions from incomparable metrics, immature cohorts, or forecasts that have not been evaluated on future periods.

This application demonstrates a reproducible analytics workflow that brings these sources together, defines metrics explicitly, and makes data quality visible. It helps a product team investigate engagement, conversion, churn, revenue, acquisition, and experiment impact.

2 What the Application Does

- Generates a deterministic sample dataset for a subscription app: users, events, subscriptions, acquisition spend, and experiment assignments.
- Transforms CSV inputs into a three-layer DuckDB warehouse (`raw`, `clean`, `mart`), preserving the source layer and removing exact duplicate rows from the clean layer.
- Checks required tables, duplicate keys, required fields, impossible dates, and missing or inconsistent invoices.
- Presents an interactive dashboard with date, channel, country, and device filters, KPIs, time series, and cohort analysis.
- Compares baseline forecasts (seasonal naive, moving average, and linear trend) using temporal backtesting and MAE/MAPE metrics.
- Analyses an A/B test using mature conversions, confidence intervals, a statistical test, power, and minimum detectable effect (MDE).

3 Requirements

3.1 Functional Requirements

1. Generate reproducible data from a configurable random seed.
2. Build and query a local analytical warehouse.

3. Flag anomalies that could invalidate metric interpretation.
4. Calculate product metrics and segment the dashboard using shared filters.
5. Compare forecasting methods with chronological backtests that avoid temporal leakage.
6. Communicate experiment uncertainty and make recommendations proportionate to the evidence.

3.2 Non-functional Requirements and Constraints

- Run locally with Python 3.11+, Streamlit, and DuckDB; no cloud service is required.
- Keep generation, warehouse, quality, metrics, forecasting, experimentation, and presentation in separate modules.
- Keep the pipeline and analyses deterministic for reproducibility and review.
- Use synthetic data only; no personal information or real company data is included.

4 Metrics and Assumptions

DAU/WAU/MAU count distinct active users over rolling 1/7/30-day windows. Trial-to-paid conversion includes only trials whose full seven-day observation window has elapsed. Period churn divides effective cancellations by paid subscriptions active at the beginning of the period. Cohort retention considers a user active when they have at least one event in a month. LTV is an estimate based on monthly ARPU, an assumed 80% gross margin, and estimated monthly churn; it is not an observed value. Acquisition spend in filtered segments is allocated in proportion to selected signups for the corresponding channel and day.

The demo simplifies monthly billing, prices, and acquisition costs. It does not model refunds, taxes, app-store fees, currencies, or reactivations. Forecasts are interpretable baselines, not promises of future performance.

5 Architecture

Component	Responsibility
Generator	Deterministic synthetic data and scenario consistency
DuckDB warehouse	Raw/clean/mart layers and analytical facts/dimensions
Data quality checks	Explicit validations and failure samples
Metrics and analysis	KPIs, retention, forecast backtests, A/B inference
Streamlit	Filters, charts, tables, and decision-focused summaries

6 Example Results Interpretation

With the default seed 42, the warehouse contains 5,000 users, 665,480 clean events, and 9,276 synthetic invoices. In the onboarding experiment, among users with a mature observation window, control conversion is 59.13% (n=2,420) and treatment conversion is 60.30% (n=2,373): absolute lift +1.17 percentage points, 95% CI from −1.61 to +3.94 points, $p = 0.409$, estimated power 13.1%, and MDE of about 3.94 points.

The demo recommendation is to extend the experiment before deciding: the current sample cannot reliably distinguish a useful effect from no effect. This is an illustrative recommendation based on simulated data, not a conclusion about a real product.

7 Run Instructions

From the project directory:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e '.[dev]'
streamlit run app.py
```

If the DuckDB warehouse is missing, the app generates the default synthetic dataset and builds the warehouse on first launch. To build it explicitly before starting the dashboard, run:

```
python -m subscription_analytics.cli build --output data --seed 42 --users 5000 --days 180
```

8 Cloud Deployment

The project can be deployed from GitHub using Streamlit Community Cloud. Select the repository, branch `main`, and entrypoint `app.py`. The deployment installs runtime dependencies from `requirements.txt`; on first launch, the app creates its synthetic demo data and DuckDB warehouse automatically. No secrets or external services are required.

The Bending Spoons logo is included to identify the application context; it is a trademark of its respective owner. The vector artwork was obtained from the official website: https://bendingspoons.com/.